

챕터 1. 리플렉션

오픈이는 이제 막 자바(Java)를 배우기 시작한 학생입니다. 이번 학기 과제인 **게시판 웹 서버**를 만들기 위해 호기롭게 코드를 짜기 시작했지만, 몇 줄 쓰지도 못하고 벽에 부딪혔습니다.

문제는 게시판 기능이 아니라 그 이전 단계에 있었습니다. 사용자의 요청을 처리하려면, 어떤 주소로 들어왔는지 문자열을 일일이 잘라내어 파악하고 주소마다 실행할 코드를 짚지어주는 작업이 먼저 필요했습니다.

정작 게시글 화면은 하나도 만들지 못했는데, 요청을 분류하기 위한 분기문(if-else)만 에너지를 쏟고 있었습니다. 오픈이는 멍하니 모니터를 보며 한숨을 쉬었습니다.

과제는 게시판인데, 서버 뼈대만 만들다 학기가 끝나겠네.

다음 날, 오픈이는 같은 수업을 듣는 선배에게 사정을 털어놓았습니다. 선배는 오픈이가 지금까지 쓴 코드를 훑어보더니 말했습니다.

선배 : "이런 걸 처음부터 직접 만드는 사람은 없어요. 웹 서버에 공통으로 필요한 뼈대는 프레임워크에 이미 갖춰져 있거든요. 자바 웹이면 보통 스프링(Spring)을 쓰니까, 그 위에 만들고 싶은 게시판 기능만 구현해 봐요."



그림 1-1. 챕터 1의 실습 흐름

소스코드 준비

이번 챕터의 코드는 스프링 프로젝트가 아니라 순수 자바입니다. 빌드 도구 없이 IDE에서 각 `App.java` 의 `main()` 메서드를 실행합니다.

앞의 소스코드 준비에서 클론한 예제 저장소로 이동합니다.

터미널 **챕터 1 폴더로 이동**

```
cd spring-start/ch01
```

ch01 폴더는 패키지 **com.reflection** 아래 세 단계로 나뉩니다.

ch01 파일 구조

```
spring-start/ch01/src/main/java/com/reflection/
├─ ex01/
│   ├── App.java          # [작성] if-else 라우팅
│   └─ BoardController.java # [작성] insert·delete·update 메서드
├─ ex02/
│   ├── App.java          # [작성] 리플렉션으로 호출
│   ├── BoardController.java # [작성] 표식 붙은 컨트롤러
│   └─ RequestMapping.java # [작성] 커스텀 어노테이션
└─ ex03/
    ├── App.java          # [작성] 클래스로더로 폴더 스캔
    ├── BoardController.java # [작성] @Controller 부착
    ├── Controller.java    # [작성] 타입 어노테이션
    └─ RequestMapping.java # [참고] ex02와 동일
```

핵심 로직 자리는 비어 있습니다. 챕터를 따라 코드를 채우고, 막히면 **spring-end**의 완성 코드를 참고하세요.

1.1 스프링의 실행 원리

1.1.1 프레임워크

어떤 웹 서버든 사용자의 요청을 받고, 주소를 확인해 알맞은 코드로 넘기는 과정이 반드시 필요합니다. 순수 자바만으로 개발할 때는 이 공통 작업을 개발자가 일일이 직접 구현해야 했습니다.

하지만 스프링 같은 **프레임워크(Framework)**는 이런 뼈대를 이미 갖춘 상태로 제공합니다. 건축에 비유하자면 기둥과 배관 공사가 모두 끝난 건물과 같습니다. 개발자는 바닥부터 건물을 새로 올릴 필요 없이, 완성된 뼈대 안에서 원하는 기능만 채워 넣으면 됩니다.

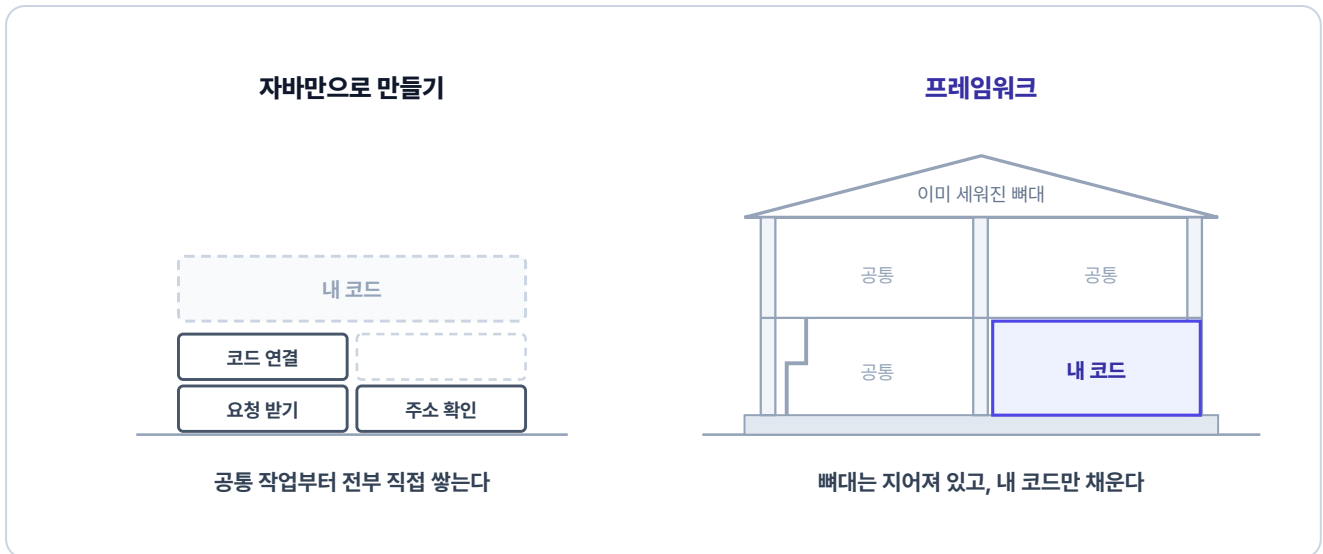


그림 1-2. 자바와 프레임워크의 차이

프레임워크에서는 개발의 출발선이 달라집니다. 번거로운 초기 과정을 건너뛰고 곧바로 핵심 기능을 구현할 수 있습니다. 웹 개발에 필수적인 로그인이나 데이터베이스 연동 역시 이미 검증된 형태로 제공되어 개발 시간을 크게 단축해 줍니다.

또한, 프레임워크는 코드를 배치하는 규칙을 정해둡니다. 일관된 규칙이 적용된 덕분에 다른 사람이 작업한 프로젝트를 열어봐도 전체적인 구조와 역할을 쉽게 파악할 수 있습니다.

1.1.2 제어의 역전

프레임워크의 가장 큰 특징은 코드의 호출 방향이 뒤바뀐다는 점입니다. 순수 자바만으로 개발할 때는 개발자가 전체 흐름을 통제하며 필요한 기능을 직접 호출합니다. 하지만 프레임워크 환경에서는 그 반대입니다. 개발자가 정해진 규칙에 맞춰 코드를 준비해 두면, 프레임워크가 필요한 시점에 개발자의 코드를 호출합니다.



그림 1-3. 호출 방향의 역전

이처럼 프로그램의 실행 주도권이 개발자에서 프레임워크로 뒤바뀌는 현상을 **제어의 역전(IoC, Inversion of Control)** 이라고 합니다.

스프링 역시 이 원칙을 따릅니다. 요청을 받아 담당 코드로 넘기는 일부터 필요한 객체를 만들어 연결하는 일까지, 애플리케이션이 돌아가는 데 필요한 공통 작업을 스프링이 전담합니다. 개발자는 각 요청에 어떤 결과를 응답할지 핵심 기능에만 집중할 수 있습니다.

1.1.3 리플렉션

주도권이 프레임워크로 넘어갔다는 것은, 개발자가 작성한 코드를 스프링이 대신 실행해 준다는 뜻입니다. 하지만 여기에는 한 가지 큰 제약이 있습니다.

보통의 자바 프로그램에서는 개발자가 호출할 대상을 코드에 직접 명시합니다. 자신이 만든 클래스이므로 이름을 정확히 알고 있고, `new` 키워드로 객체를 생성해 필요한 메서드를 직접 적어두면 됩니다.

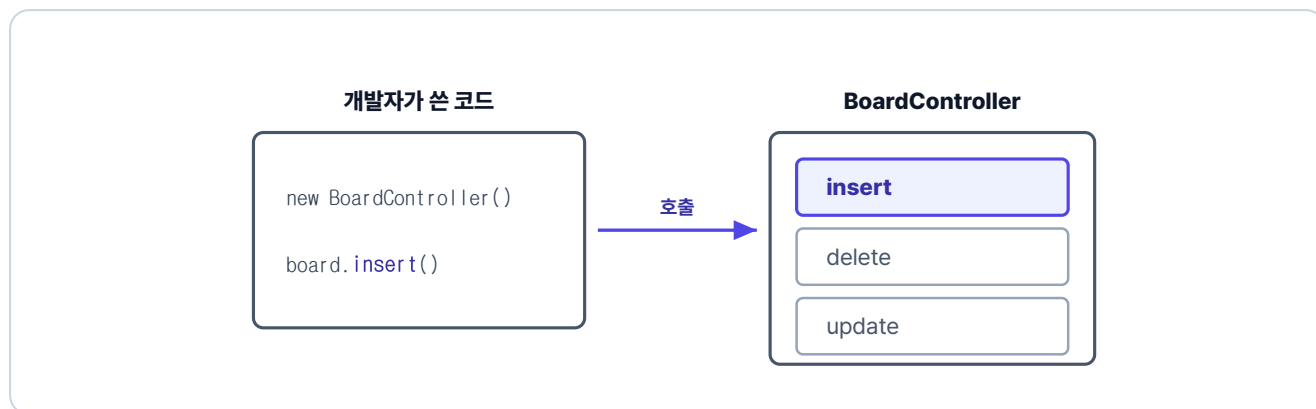


그림 1-4. 자바의 직접 호출

반면 스프링은 개발자가 미래에 어떤 클래스와 메서드를 만들지 미리 알 수 없습니다. 그런데도 이름조차 모르는 코드를 실행 중에 찾아내 호출해야 합니다.

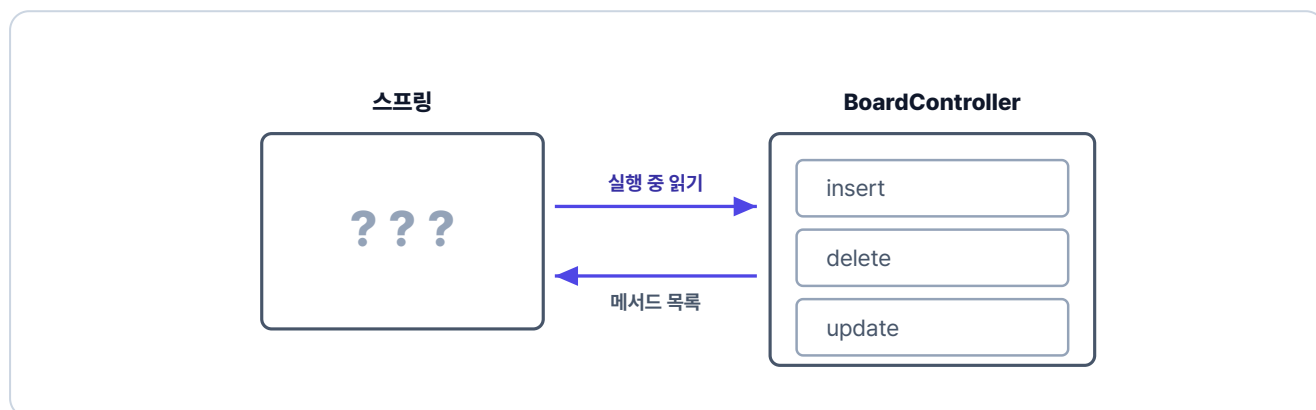


그림 1-5. 스프링의 메서드 탐색

이러한 모순을 해결해 주는 기술이 바로 **리플렉션(Reflection)** 입니다. 리플렉션은 프로그램이 실행되는 도중(런타임)에 클래스와 메서드 정보를 읽어내고, 객체 생성과 호출까지 할 수 있는 자바의 핵심 기능입니다.

1.2 리플렉션 구현

리플렉션이 실제로 어떻게 동작하는지 자바 코드로 직접 구현해 보겠습니다. 가장 단순한 형태의 코드부터 시작해 한 단계씩 기능을 확장해 나갑니다.

1.2.1 if-else 라우팅

가장 기본적인 요청 처리 방식은 들어온 요청 주소를 if 문으로 비교하여 알맞은 메서드를 직접 호출하는 것입니다. 이를 확인하기 위해 먼저 게시글을 다루는 **BoardController**에 등록(insert), 수정(update), 삭제(delete) 메서드를 작성하겠습니다. 각 메서드는 호출될 때 콘솔에 자신의 이름을 출력합니다.

`ex01/BoardController.java` 를 열어 아래와 같이 작성합니다.

실습 1 ex01/BoardController.java. 요청을 처리할 컨트롤러

```
public class BoardController {  
  
    public void insert() {  
        System.out.println("insert 호출됨");  
    }  
  
    public void delete() {  
        System.out.println("delete 호출됨");  
    }  
  
    public void update() {  
        System.out.println("update 호출됨");  
    }  
}
```

이제 요청 주소를 알맞은 담당 메서드로 연결해 주는 **라우팅(Routing)** 처리를 위해 `main()` 메서드를 작성해 보겠습니다.

`ex01/App.java` 를 열어 아래와 같이 작성합니다.

실습 2 ex01/App.java. 주소에 따라 메서드 호출

```
public class App {
    public static void main(String[] args) {

        String uri = "/insert";
        BoardController boardController = new BoardController();

        // TODO: uri에 맞는 메서드를 직접 호출한다
        // 1. 주소가 /insert면 insert() 호출
        if (uri.equals("/insert")) {
            boardController.insert();
        }
        // 2. /update면 update() 호출
        } else if (uri.equals("/update")) {
            boardController.update();
        }
        // 3. /delete면 delete() 호출
        } else if (uri.equals("/delete")) {
            boardController.delete();
        }
    }
}
```

uri가 /insert이므로 실행하면 insert()가 호출됩니다.

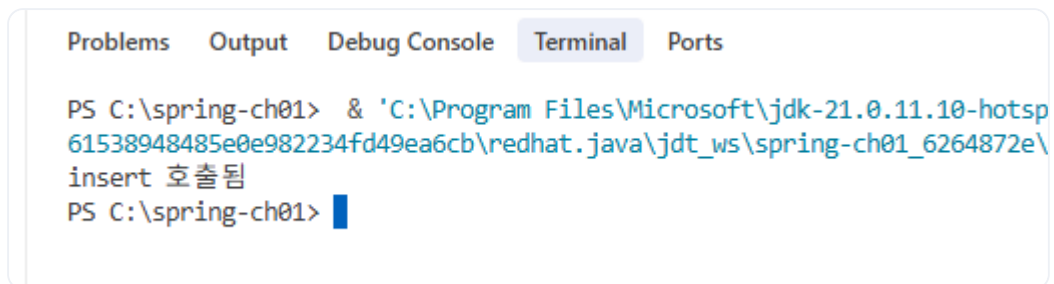


그림 1-6. if 문 라우팅 실행 결과

여기서 만약 게시글을 조회하는 select() 메서드를 하나 더 추가한다면 어떨까요?

참고 ex01/BoardController.java. 메서드 추가

```
public void select() {
    System.out.println("select 호출됨");
}
```

컨트롤러에 `select()` 메서드를 추가했지만, 정작 `main()` 메서드에는 `/select` 주소를 처리할 분기문이 없습니다. 이 방식은 새로운 메서드를 추가할 때마다 매번 `else if` 문을 추가해야 합니다. 이처럼 컨트롤러뿐만 아니라 호출부의 코드까지 함께 수정해야 한다는 점이 이 구조의 가장 큰 한계입니다.

1.2.2 어노테이션과 리플렉션

앞서 확인한 한계를 극복하려면, 호출부 코드를 매번 수정하지 않고도 프로그램이 실행 중에 알맞은 메서드를 찾아 실행하는 구조가 필요합니다. 이때 사용하는 것이 어노테이션과 리플렉션입니다.

방식은 이렇습니다. 주소를 `if` 문에 일일이 나열하는 대신, 각 메서드 위에 담당할 주소를 표식으로 붙여 둡니다. 클래스나 메서드에 붙이는 이 표식을 **어노테이션(Annotation)** 이라고 합니다.

그리고 프로그램이 실행될 때 컨트롤러의 메서드를 모두 확인하여 이 표식을 읽고, 요청 주소와 일치하는 메서드를 찾아 호출합니다. 실행 중에 이렇게 코드를 분석하고 알맞은 메서드를 호출하는 일을 리플렉션이 담당합니다.

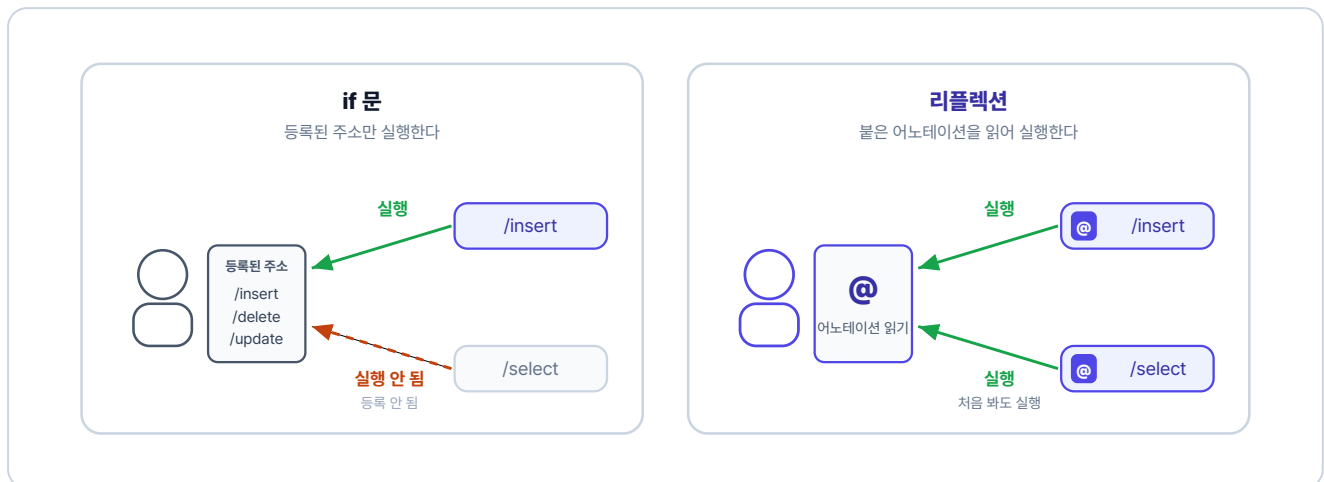


그림 1-7. if 문과 리플렉션의 차이

이제 각 메서드가 담당할 요청 주소를 지정하는 **@RequestMapping** 어노테이션을 직접 만들어 보겠습니다. 어노테이션은 클래스나 인터페이스와 마찬가지로 `@interface` 키워드를 사용하여 정의합니다.

`ex02/RequestMapping.java` 를 열어 아래와 같이 작성합니다.

실습 3 ex02/RequestMapping.java. 주소를 담는 커스텀 어노테이션

```
// 1. 컴파일한 뒤에도 정보를 남겨, 실행 중에 리플렉션이 이 어노테이션을 읽을 수 있게 한다
@Retention(RetentionPolicy.RUNTIME)
// 2. 붙일 수 있는 대상을 메서드로 제한한다
```

```
@Target(ElementType.METHOD)
// 3. 스프링이 제공하는 @RequestMapping과 이름만 같은 어노테이션을 직접 정의한다
public @interface RequestMapping {
    // 4. 메서드가 담당할 요청 주소를 이 속성에 적어 둔다
    String uri();
}
```

그리고 **BoardController**의 각 메서드에 어노테이션을 추가합니다.

`ex02/BoardController.java`를 열어 아래와 같이 변경합니다.

실습 4 ex02/BoardController.java. 메서드마다 담당 주소를 어노테이션으로

```
public class BoardController {

    @RequestMapping(uri = "/insert")
    public void insert() {
        System.out.println("insert 호출됨");
    }

    @RequestMapping(uri = "/delete")
    public void delete() {
        System.out.println("delete 호출됨");
    }

    @RequestMapping(uri = "/update")
    public void update() {
        System.out.println("update 호출됨");
    }

    @RequestMapping(uri = "/select")
    public void select() {
        System.out.println("select 호출됨");
    }
}
```

이어서 `main()` 메서드에서 어노테이션을 읽을 수 있도록 수정합니다.

`ex02/App.java`를 열어 아래와 같이 작성합니다.

실습 5 ex02/App.java. 어노테이션을 읽어 맞는 메서드를 호출

```
public class App {
    public static void main(String[] args) {
```

```

String uri = "/update";
BoardController boardController = new BoardController();

// TODO: 어노테이션을 읽어 uri가 일치하는 메서드를 실행한다
// 1. 컨트롤러가 가진 모든 메서드를 꺼낸다
Method[] methods = boardController.getClass().getDeclaredMethods();
for (Method method : methods) {
    // 2. 그 메서드에 붙은 @RequestMapping 어노테이션을 읽는다
    RequestMapping rm = method.getDeclaredAnnotation(RequestMapping.class);
    // 3. 어노테이션의 uri가 요청 주소와 같으면 그 메서드를 실행한다
    if (rm.uri().equals(uri)) {
        try {
            method.invoke(boardController);
            break;
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
}
}
}

```

이제 `main()` 메서드에서는 컨트롤러에 어떤 메서드가 있는지 알지 못해도, 리플렉션이 실행 중에 어노테이션을 읽어 요청 주소와 맞는 메서드를 호출합니다.

```

PS C:\spring-ch01> c:; cd 'c:\spring-ch01'; & 'C:\Program F
User\workspaceStorage\30d61538948485e0e982234fd49ea6cb\redha
update 호출됨
PS C:\spring-ch01> 

```

그림 1-8. 리플렉션 실행 결과

이러한 구조 덕분에 새 메서드에 어노테이션만 추가해 두면, 호출부 수정 없이도 리플렉션이 이를 읽고 담당 메서드를 찾아 호출할 수 있습니다.

1.2.3 컴포넌트 스캔

현재 구조는 `main()` 메서드에서 개발자가 직접 컨트롤러 객체를 생성하고 있습니다. 컨트롤러가 하나일 때는 괜찮지만, 게시판이나 회원처럼 기능이 늘어나면 매번 객체 생성 코드를 추가해야 합니다. 객체 생성을 개발자가 직접 관리하면 프로그램이 어떤 컨트롤러를 가지고 있는지 스스로 파악할 수 없어, 결국 코드를 계속 수정해야 하는 한계에 부딪힙니다.

이를 극복하려면 개발자가 직접 객체를 만드는 대신, 프로그램 실행 중에 어노테이션이 붙은 클래스를 찾아 자동으로 객체를 생성하는 구조가 필요합니다. 이러한 기능을 **컴포넌트 스캔(Component Scan)** 이라고 부릅니다.

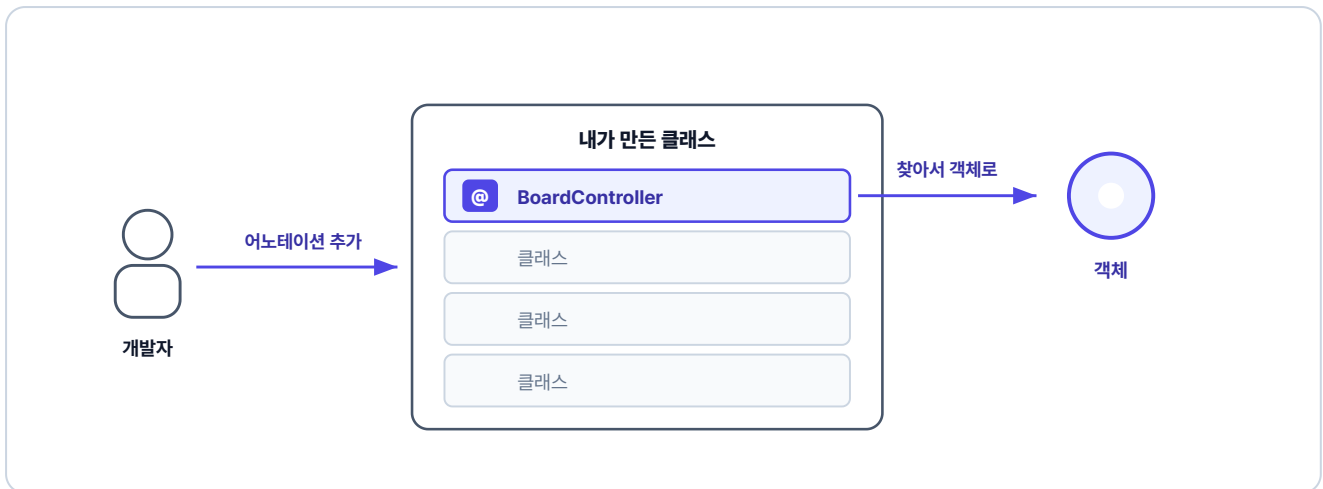


그림 1-9. 컴포넌트 스캔의 객체 생성

이번에는 메서드가 아니라 클래스에 붙일 어노테이션이 필요합니다. 컨트롤러 역할을 하는 클래스임을 나타내는 **@Controller**를 정의합니다.

`ex03/Controller.java`를 열어 아래와 같이 작성합니다.

실습 6 ex03/Controller.java. 컨트롤러임을 나타내는 어노테이션

```
// 1. 컴파일한 뒤에도 정보를 남겨, 실행 중에 리플렉션이 이 어노테이션을 읽을 수 있게 한다
@Retention(RetentionPolicy.RUNTIME)
// 2. 붙일 수 있는 대상을 클래스와 인터페이스로 제한한다
@Target(ElementType.TYPE)
public @interface Controller {
}
```

이어서 `ex03/BoardController.java`를 열어 클래스 선언 위에 **@Controller**를 추가합니다.

실습 7 ex03/BoardController.java. 클래스 위에 @Controller 어노테이션

```
@Controller
public class BoardController {

    @RequestMapping(uri = "/insert")
    public void insert() { System.out.println("insert 호출됨"); }

    @RequestMapping(uri = "/delete")
```

```

public void delete() { System.out.println("delete 호출됨"); }

@RequestMapping(uri = "/update")
public void update() { System.out.println("update 호출됨"); }

@RequestMapping(uri = "/select")
public void select() { System.out.println("select 호출됨"); }

@RequestMapping(uri = "/create")
public void create() { System.out.println("create 호출됨"); }
}

```

이제 핵심이 되는 `main()` 메서드를 살펴보겠습니다. 코드를 보면 직접 객체를 생성하는 구문은 어디에도 없습니다. 대신 `ex03` 폴더의 실제 경로를 읽고 그 안의 클래스 파일을 하나씩 확인하여, **@Controller** 어노테이션이 붙은 클래스만 찾아 객체로 생성합니다.

`ex03/App.java` 를 열어 아래와 같이 작성합니다.

실습 8 ex03/App.java. 폴더를 뒤져 @Controller 클래스를 등록하고 주소로 호출

```

public class App {
    public static void main(String[] args)
        throws URISyntaxException, ReflectiveOperationException {
        // TODO: ex03 폴더를 뒤져 @Controller 클래스만 객체로 만든다
        // 1. ex03 폴더의 실제 경로를 찾는다
        ClassLoader classLoader = Thread.currentThread().getContextClassLoader();
        URL packageUrl = classLoader.getResource("com/reflection/ex03");
        File ex03 = new File(packageUrl.toURI());

        List<Object> instances = new ArrayList<>();

        // 2. 폴더 안의 .class 파일을 하나씩 살펴본다
        for (File file : ex03.listFiles()) {
            System.out.println("파일명: " + file.getName());
            if (file.getName().endsWith(".class")) {
                String className = "com.reflection.ex03."
                    + file.getName().replace(".class", "");
                Class<?> cls = Class.forName(className);
                // 3. @Controller 어노테이션이 붙은 클래스만 객체로 만들어 보관한다
                if (cls.isAnnotationPresent(Controller.class)) {
                    System.out.println("어노테이션이 있는 클래스 : " + file.getName());
                    Object instance = cls.getDeclaredConstructor().newInstance();
                    instances.add(instance);
                }
            }
        }
    }
}

```

```

        System.out.println("-----");

        // 4. 보관한 객체에서 주소에 맞는 메서드를 찾아 실행한다
        findUri(instances, "/delete");
    }

    // 5. 보관한 객체를 하나씩 확인해 주소가 일치하는 메서드를 실행한다
    public static void findUri(List<Object> instances, String uri) {
        for (Object instance : instances) {
            Method[] methods = instance.getClass().getDeclaredMethods();
            for (Method method : methods) {
                RequestMapping rm = method.getDeclaredAnnotation(RequestMapping.class);

                if (rm.uri().equals(uri)) {
                    try {
                        method.invoke(instance);
                        break;
                    } catch (Exception e) {
                        e.printStackTrace();
                    }
                }
            }
        }
    }
}

```

`getDeclaredConstructor().newInstance()` 는 클래스의 기본 생성자를 찾아 객체를 만듭니다. 리플렉션으로 클래스와 생성자를 다루는 동안 생기는 예외는 **ReflectiveOperationException** 하나로 묶어 선언했습니다.

실행하면 콘솔에 `ex03` 폴더 안의 파일 이름이 하나씩 출력되고, 그중 **@Controller**가 붙은 클래스가 객체로 등록됩니다. 이어서 `findUri()` 가 등록된 객체에서 `/delete`와 주소가 일치하는 `delete()` 를 찾아 호출합니다.

```

파일명: App.class
파일명: BoardController.class
어노테이션이 있는 클래스 : BoardController.class
파일명: Controller.class
파일명: RequestMapping.class
-----
delete 호출됨

```

그림 1-10. 컴포넌트 스캔 실행 결과

이제 개발자는 클래스에는 **@Controller**를, 메서드에는 **@RequestMapping**을 추가하면 됩니다. 객체를 만드는 일도, 메서드를 찾아 호출하는 일도 프로그램이 대신 처리합니다.

1.3 객체 생성과 연결

1.3.1 빈과 컨테이너

스프링 프레임워크의 가장 핵심적인 역할은 개발자 대신 객체를 생성하고 관리하는 것입니다. 애플리케이션이 실행되면 스프링은 컴포넌트 스캔을 통해 어노테이션이 붙은 클래스를 찾고, 리플렉션을 활용해 자동으로 객체를 생성합니다.

이때 만들어진 객체들을 모아두고 관리하는 전용 공간을 **스프링 컨테이너(Spring Container)**, 컨테이너 안에 보관된 객체를 **빈(Bean)** 이라고 부릅니다.

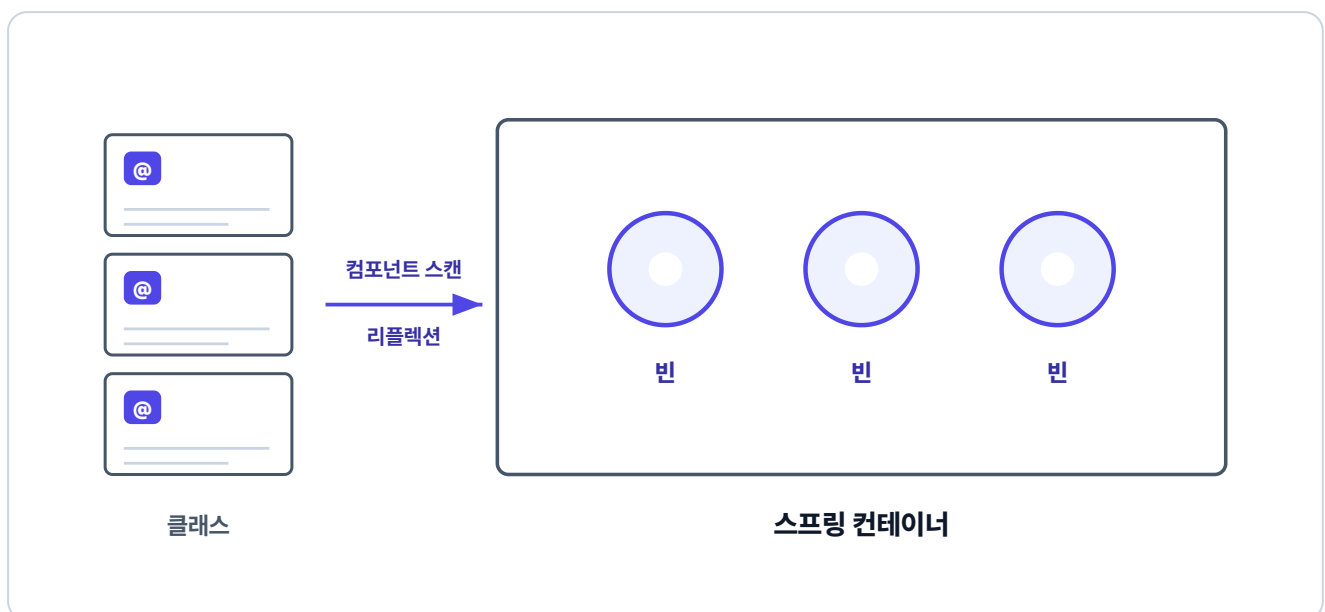


그림 1-11. 스프링 컨테이너와 빈

1.3.2 의존성 주입

그렇다면 이렇게 만들어진 빈들은 어떻게 서로 기능을 주고받을까요? 한 객체가 다른 객체의 기능을 필요로 하는 상태를 **의존성을 가진다**고 합니다. 이때 필요한 객체를 직접 생성해서 쓰면 객체 간의 결합도가 높아져, 나중에 코드를 고치거나 테스트하기가 매우 까다로워집니다.

스프링은 이 문제를 **의존성 주입(Dependency Injection)** 으로 해결합니다. 개발자가 필요한 객체를 직접 만들지 않고 코드에 선언만 해두면, 컨테이너가 실행될 때 알맞은 빈을 찾아 넣어 주는 방식입니다.

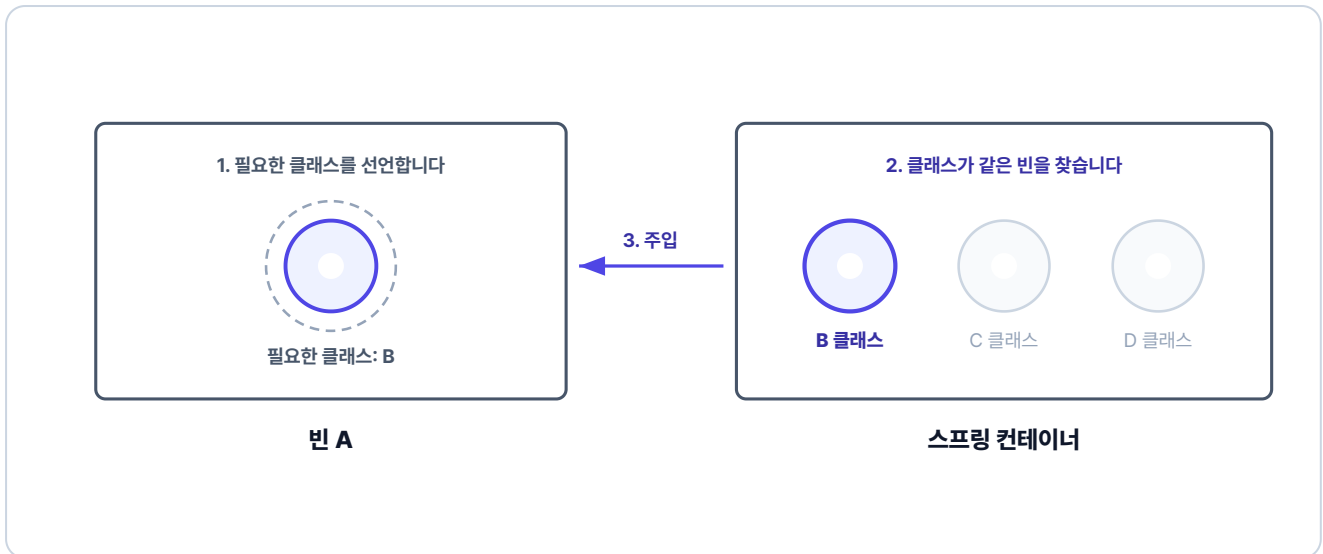


그림 1-12. 컨테이너의 빈 주입

스프링 컨테이너가 객체를 만들고 주입하는 과정을 대신해 줍니다. 덕분에 개발자는 필요한 빈을 선언만 하면 되고, 코드의 결합도 역시 낮아집니다.

1.4 스프링의 요청 흐름

하나의 요청이 서버에 도착해서 컨트롤러까지 전달되기까지 어떤 단계를 거치는지 따라가 보겠습니다.

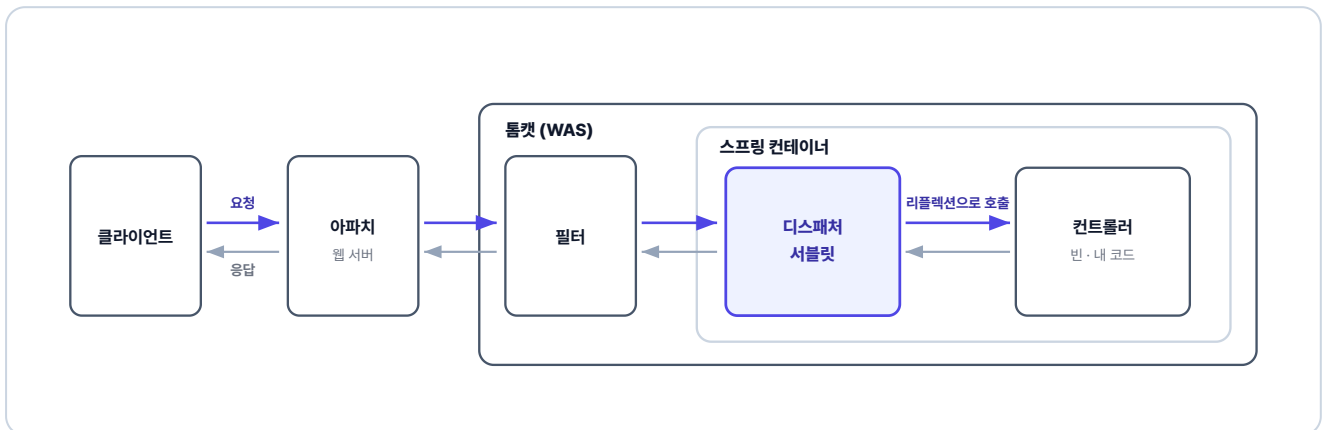


그림 1-13. 요청 처리 흐름

외부 요청이 컨트롤러로 전달되는 흐름에서 각 단계가 맡는 역할은 다음과 같습니다.

1. **웹 서버(아파치).** 클라이언트의 요청을 가장 먼저 받습니다. 이미지나 단순 HTML처럼 미리 만들어져 있는 정적인 데이터는 여기서 바로 처리하고, 자바 코드를 실행해야 하는 요청은 뒤에 있는 WAS로 넘깁니다.

2. **통캣(WAS)과 필터**. 통캣은 자바 코드를 실행해 동적인 결과를 만들어 내는 **웹 애플리케이션 서버(WAS)** 입니다. 통캣은 본격적인 처리에 앞서 요청을 필터에 통과시키고, 필터에서는 로그인 확인처럼 모든 요청에 공통으로 필요한 작업을 미리 처리합니다.
3. **디스패처 서블릿**. 필터를 거친 요청은 스프링 컨테이너 안의 **디스패처 서블릿(DispatcherServlet)** 으로 전달됩니다. 디스패처 서블릿은 요청 주소를 분석해 이 작업을 담당할 컨트롤러를 찾아냅니다.
4. **컨트롤러 호출**. 어떤 주소를 어떤 컨트롤러에 연결할지 실행 중에 찾아내는 일은 스프링이 리플렉션으로 대신 처리합니다. 따라서 개발자는 컨트롤러 코드만 작성해 두면 됩니다.

1.5 학습 지도

이 책의 학습 여정은 다섯 챕터로 이어집니다.

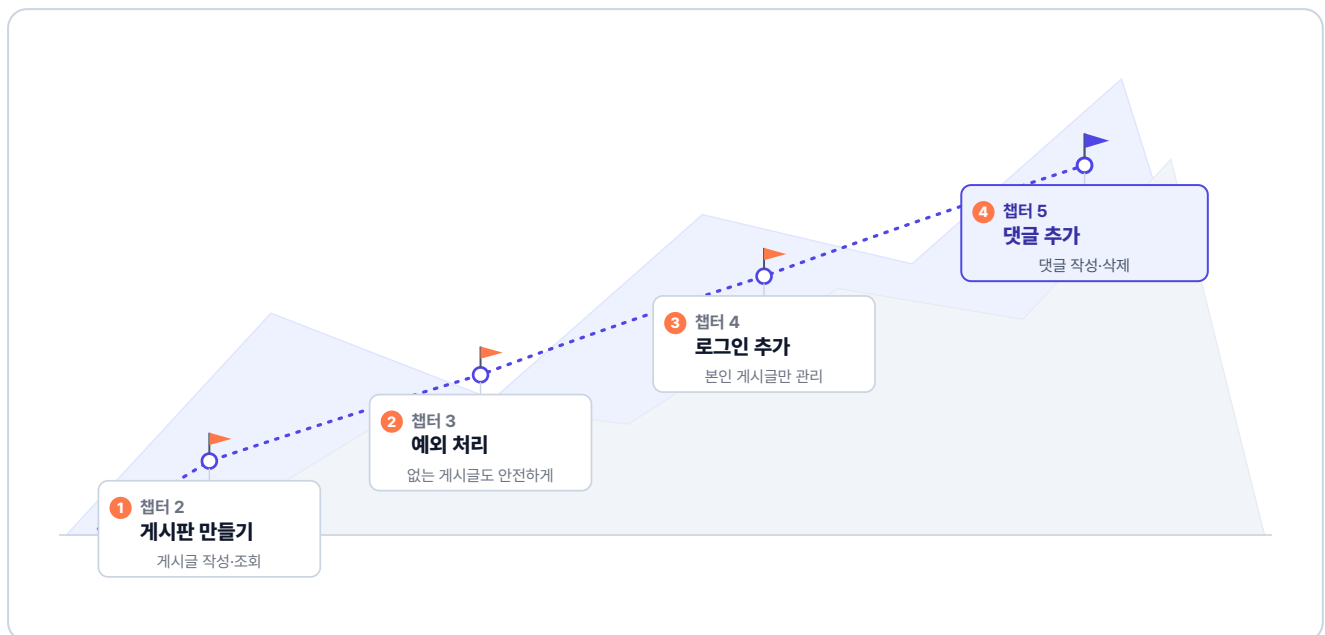


그림 1-14. 앞으로의 학습 여정

각 챕터에서 다루는 내용은 다음과 같습니다.

- 챕터 2에서는 JPA와 3계층 구조로 게시판을 만들어 게시글 등록·조회·수정·삭제 기능을 완성합니다.
- 챕터 3에서는 DTO와 전역 예외 처리를 적용해 응답 형식을 정리합니다.
- 챕터 4에서는 JWT를 활용해 로그인 기능을 추가하고, 본인만 자기 게시글을 관리하게 합니다.
- 챕터 5에서는 양방향 매핑으로 게시글과 댓글을 연결해 댓글 기능을 추가합니다.

다음 챕터에서는 컨트롤러와 서비스, 리포지토리를 직접 만들어 게시글을 저장하고 조회하는 REST API를 완성합니다.

이것만은 기억하자

- 프레임워크는 공통으로 필요한 뼈대를 미리 갖춘 틀입니다.
- 스프링은 컴포넌트 스캔으로 클래스를 찾아 객체를 등록하고, 어노테이션을 읽어 실행할 대상을 결정합니다. 이처럼 개발자가 아닌 프레임워크가 리플렉션을 통해 알맞은 메서드를 직접 찾아 실행하는 구조를 **제어의 역전**이라고 합니다.
- 스프링 컨테이너는 이렇게 생성한 객체(빈)를 보관하다가 필요한 곳에 주입해 줍니다. 개발자는 필요한 빈을 선언하기만 하면 되므로, 코드 간의 결합도가 한결 낮아집니다.
- 클라이언트의 요청은 아파치와 톰캣을 거쳐 디스패처 서블릿으로 전달됩니다. 디스패처 서블릿은 요청 주소를 분석해 담당 컨트롤러를 호출하며, 실행 중에 알맞은 대상을 찾아 연결하는 작업 역시 리플렉션이 처리합니다.